

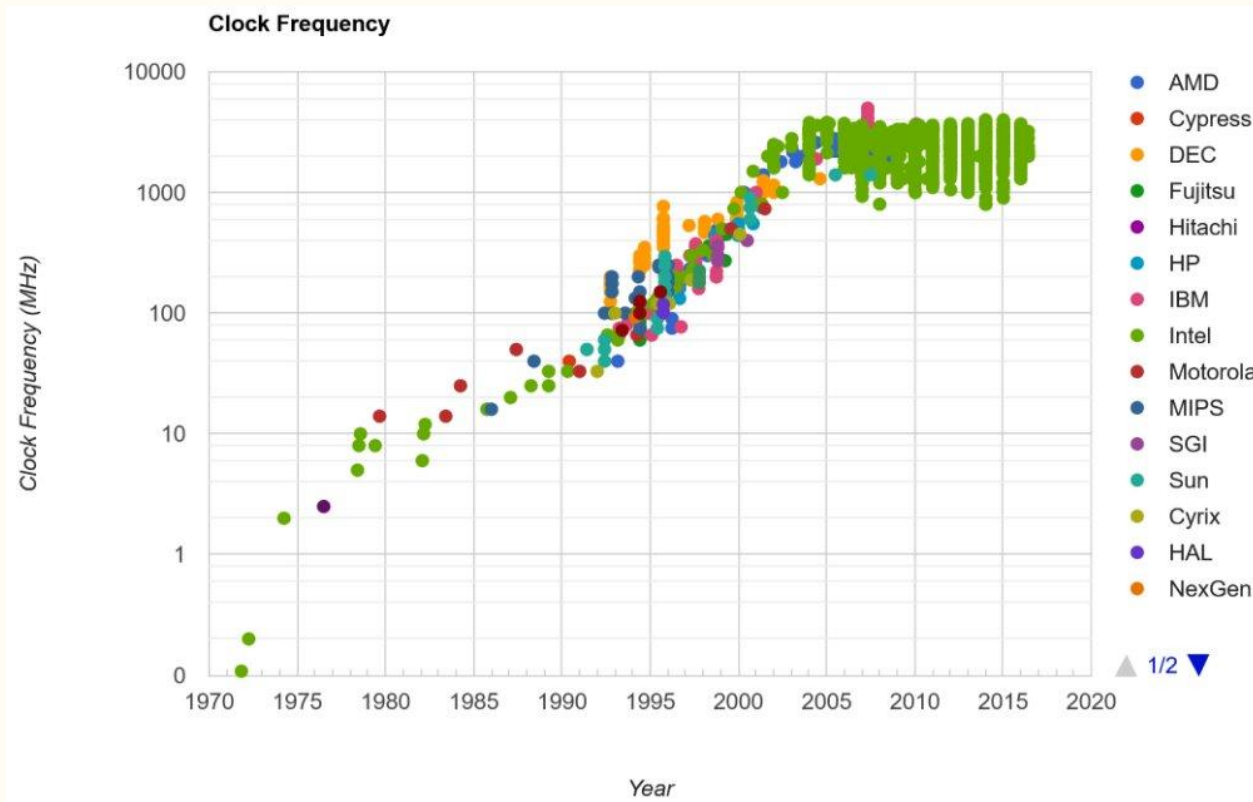
HPC Architectures

Dr David Henty

d.henty@epcc.ed.ac.uk

The fundamental issue

- CPU clock frequency has plateaued since the early 2000s.
 - This is primarily due to power limits being reached.



- Andrew Danowitz, Kyle Kelley, James Mao, John P. Stevenson, and Mark Horowitz. 2012. CPU DB: recording microprocessor history. *Commun. ACM* 55, 4 (April 2012), 55–63. <https://doi.org/10.1145/2133806.2133822>

Why HPC and parallel computing?

- Scientific simulation and modelling have driven the need for greater computing power for many decades
 - more recently, Data Science and AI
- Single-core processors cannot be made that have enough resource for the simulations needed
 - making processors with faster clock speeds is difficult due to cost and power/heat limitations
 - expensive to put huge memory on a single processor
- Severe limitations on performance of *serial* computing
- Solution: *parallel* computing
 - divide up the work among numerous connected systems

Serial vs Parallel

- Serial
 - **happening one after another in time or order**
- Parallel
 - **to happen at the same time as something else**, or be similar or equal to something else
- From <https://dictionary.cambridge.org>

Building Blocks of (Parallel) Computing



Four principal technologies which make up HPC systems:

- Processors:
 - to calculate
 - includes accelerators like GPUs
- Memory:
 - for temporary storage of data
- Storage:
 - disks/drives for storing input/output data and tapes for long term archiving of data
- Network Interconnect:
 - so processors can talk to each other (and the outside world)

What do we mean by “performance”?

- For scientific and technical programming use FLOPS
 - Floating Point OPerations per Second
 - $1.324398404 + 3.6287414 = ?$
 - $2.365873534 * 2443.3147 = ?$
- Modern supercomputers typically measured in PFLOPS (PetaFLOPS)
 - Kilo, Mega, Giga, Tera, Peta, Exa = 10^3 , 10^6 , 10^9 , 10^{12} , 10^{15}
 - several exascale computers now exist
- Other disciplines have their own performance measures
 - forecast days per day
 - training datasets analysed per second
 - frames per second, database accesses per second, ...

Processors

Basic functionality:

- execute instructions to perform arithmetic operations (integer / floating point)
- load data from memory and store data to memory
- decide which instructions to execute next

Arithmetic is performed on values in *registers*:

- local storage in the processor typical size ~100 values.
- moving data between memory and registers must be done explicitly by load and store instructions
- separate integer and floating-point registers

Basic characteristics relevant for HPC:

- Clock speed
- Peak floating-point capability (FLOPS)

Processors 2

Clock speed determines rate at which instructions executed

- modern chips are around 2-3 GHz
- integer and floating-point calculations can be done in parallel
- can also have multiple instruction, e.g. simultaneous add and multiply
- peak flop rate is clock rate x floating-point operations per clock cycle
- e.g. 2 GHz x 8 flops/cycle = 16 GFLOPS

Whole series of hardware innovations

- pipelining
- out-of-order execution, speculative execution

Details become important for extracting high performance

- most features are fairly generic

Moore's Law

“CPU power doubles every 2 years”

- Strictly speaking, applies to transistor density

Held true for ~35 years

- People have predicted its demise many times
- It hasn't happened yet but may be slowing.

Increased performance from increases in parallelism AND clock rate

- Fine grain parallelism (pipelining)
- Medium grain parallelism (superscalar, hardware multithreading)
- Coarse grain parallelism (multiple processors on a chip)
- First two seem to be (almost) exhausted: main trend is now towards multicore
- Clock speed virtually stalled due to power consumption / heat dissipation

Accelerators

Current popular trend is to include additional special purpose processors alongside the main CPU

- Large numbers of relatively simple cores or vector processors
- High memory bandwidth

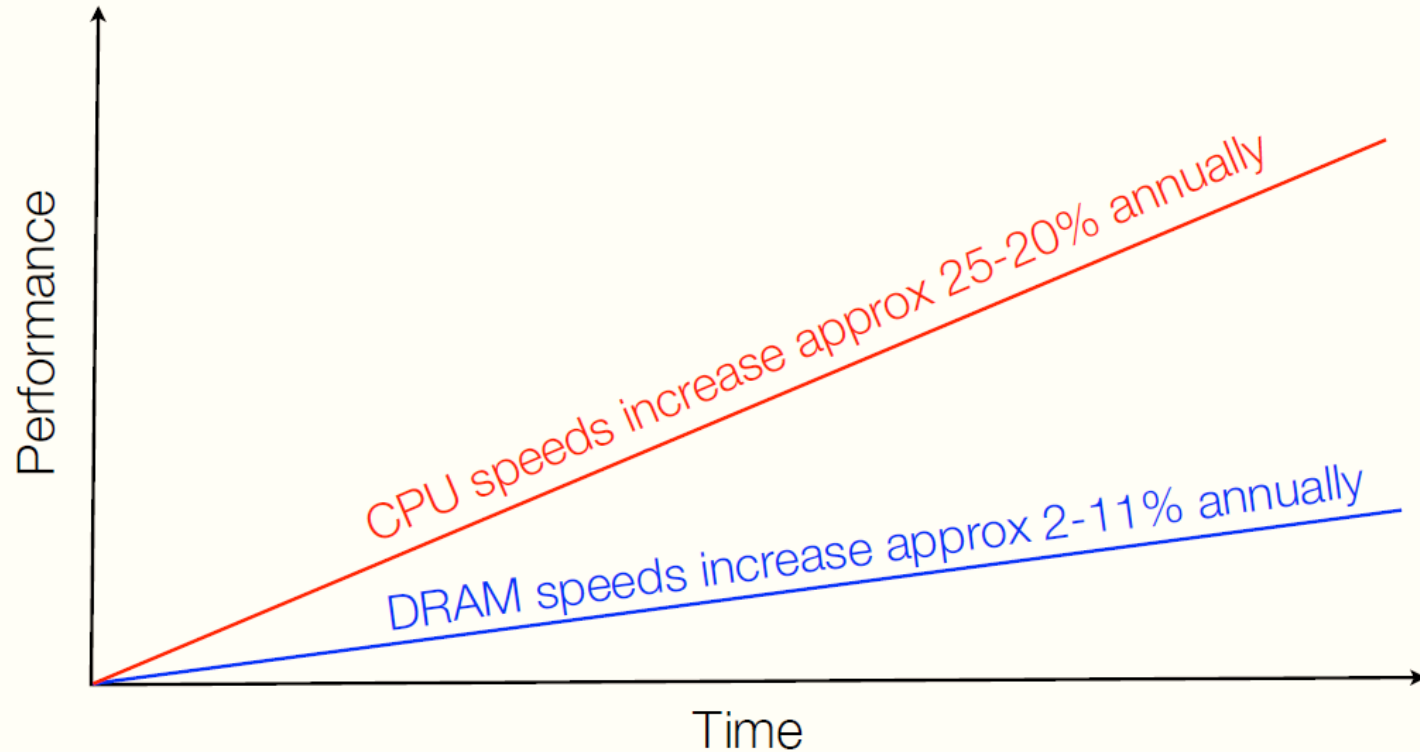
Much of current interest is focussed on GPGPUs (general purpose graphics processing units)

- Low cost due to high mass market volumes
- High bandwidth graphics memory

May not be a long-term solution

- Industry trends suggest tighter integration of simple cores onto a single piece of silicon

Performance Trend – CPU v RAM



Main Memory

Memory speed is often the limiting factor for HPC applications.

- Keeping the CPU fed with data is the key to performance.

Memory is a substantial contributor to the cost of systems.

- Typical HPC systems have a few Gbytes of memory per processor.
- Technically possible to have much more than this, but it is too expensive and power-hungry.

Basic characteristics

- Latency: how long you have to wait for data to arrive
- Bandwidth: how fast it actually comes in
- Ballpark figures: 100's of nanoseconds and tens of Gbytes/s.

Cache memory

Memory latencies are very long

- 100s of processor cycles
- fetching data from main memory over 100x slower than arithmetic

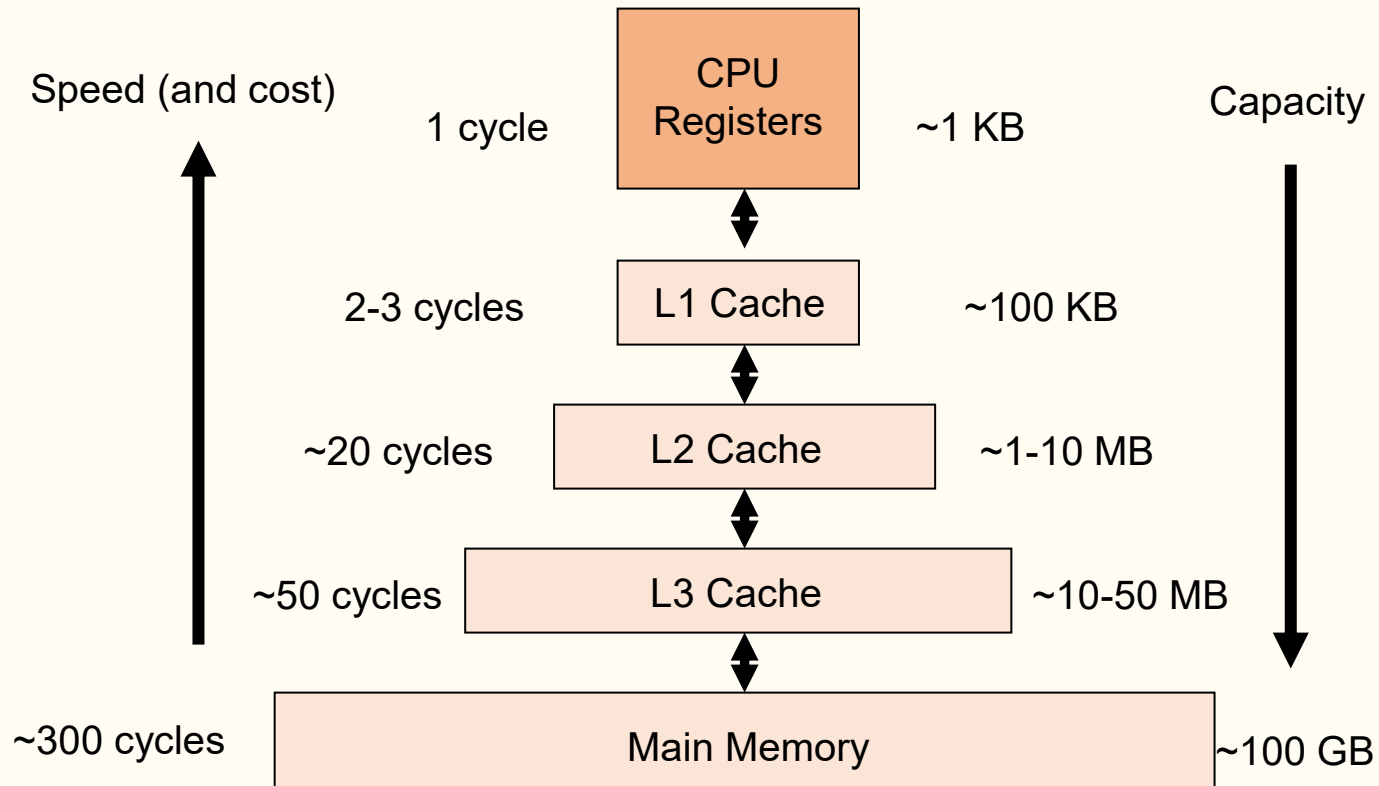
Solution: introduce cache memory

- much faster than main memory
- ...but much smaller than main memory
- ...also more expensive and power hungry.
- keeps copies of recently used data

Modern systems use a hierarchy of two or three levels of cache

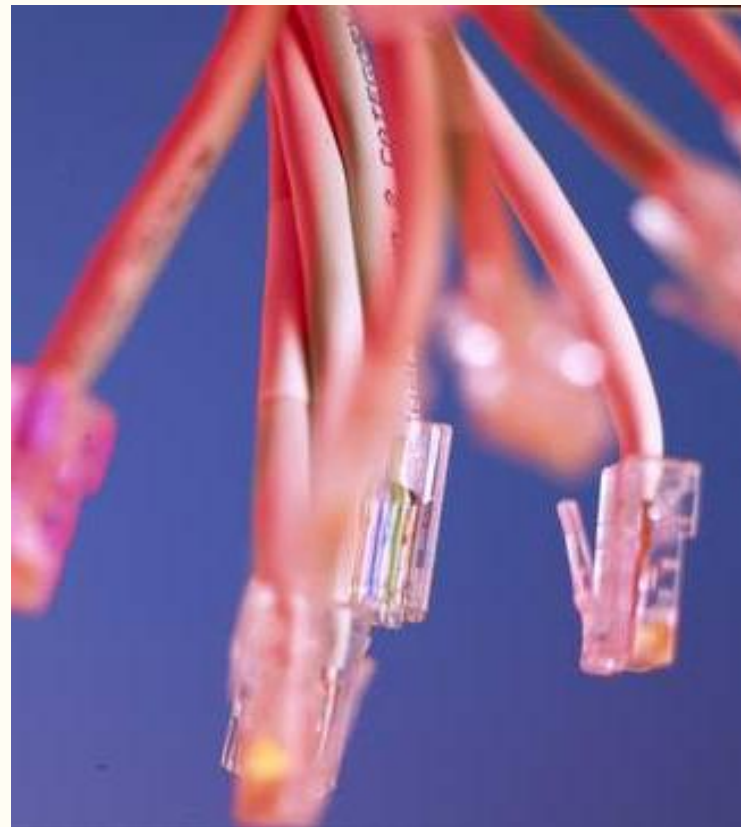
- top 2 levels typically on-chip

Memory hierarchy

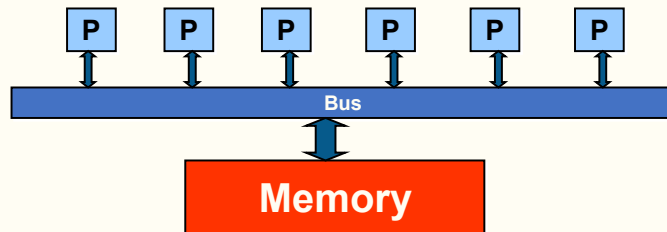


Interconnect

- CPUs need to exchange data.
- For distributed systems we connect via external hardware.
- Characterised by:
 - Latency (several microseconds)
 - Bandwidth (many GB/s)

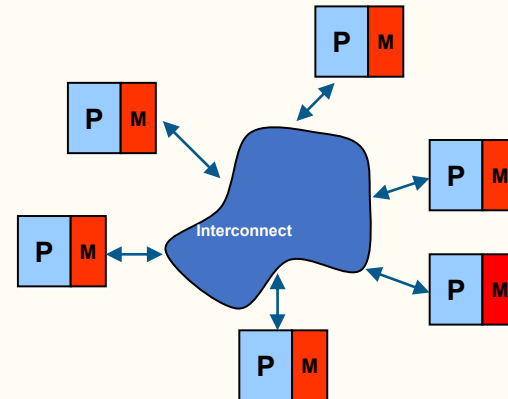


Types of HPC systems



Shared-memory: OpenMP

- Multiple processors share a single memory space (e.g. your laptop!)
- Simple to program for many problems
- Scaling up is problematic



Distributed memory: MPI

- Each processing unit has its own memory space
- Excellent scaling properties
- Can be more complex to program due to explicit communications

Accelerators (GPUs)

- Specialist processing units attached to main CPU
- Can be difficult to extract good performance for general scientific computing
- Huge growth in use of AI / Machine Learning

Distributed Shared Memory (clusters)

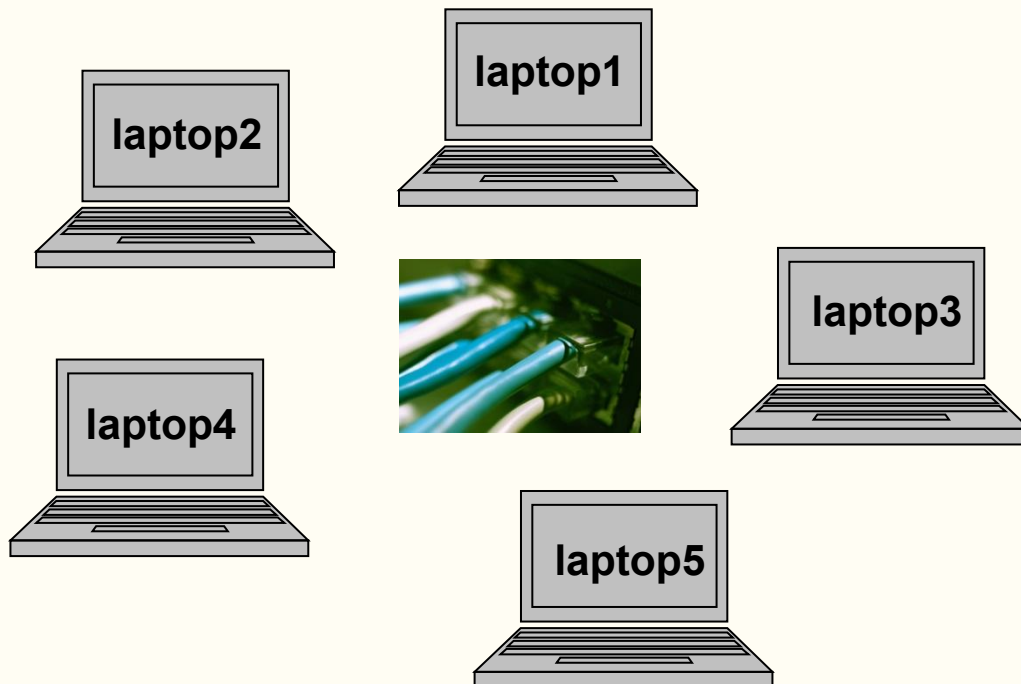
Dominant architecture is a hybrid of these two approaches:

Distributed Shared Memory.

- Due to most HPC systems now being built from commodity hardware – trend to multicore processors.
- Each Shared memory block is known as a *node*.
- Usually 64+ processors per node.
- Nodes often also contain accelerators.

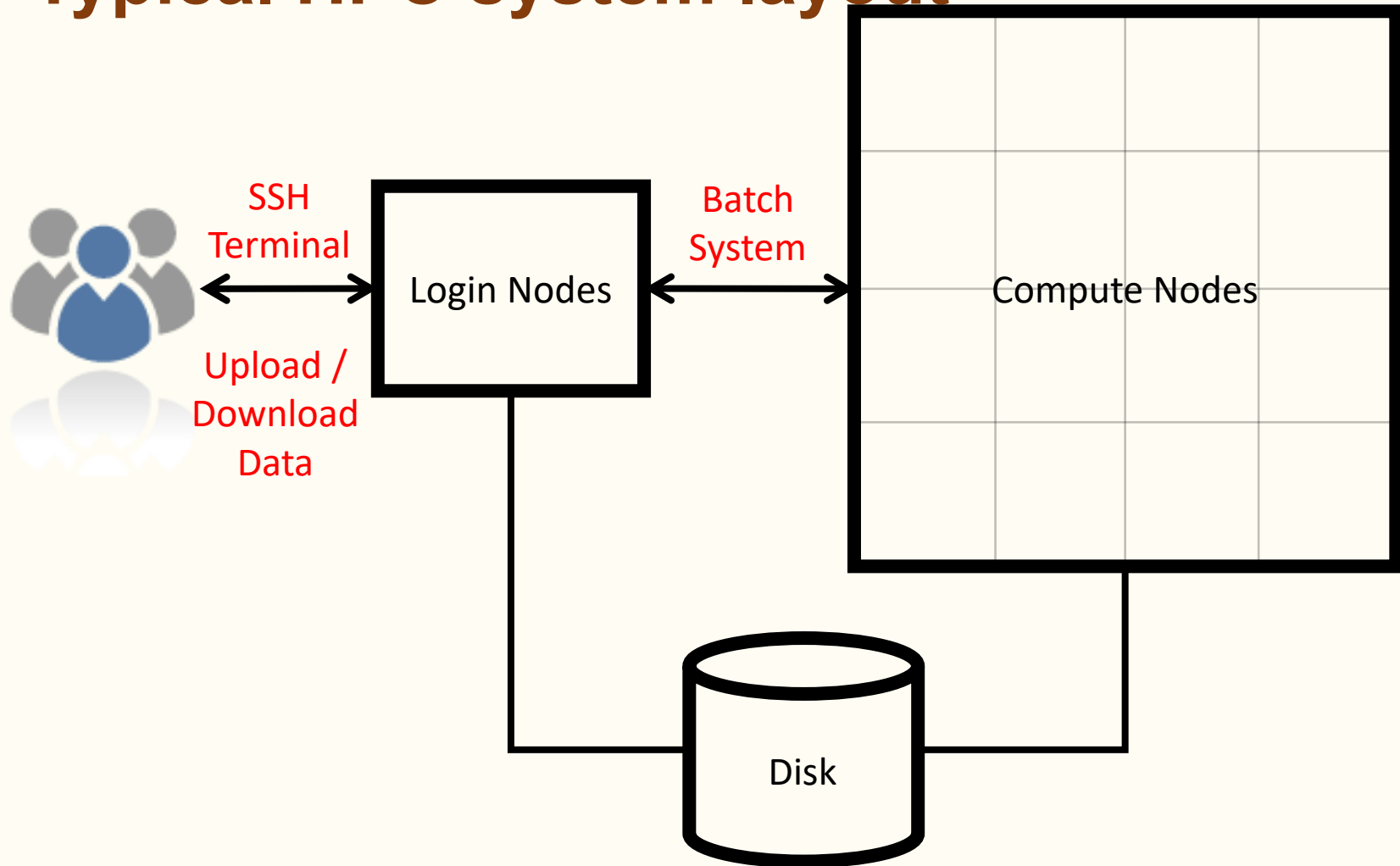
Generic Parallel Machine

- Good conceptual model is collection of multicore laptops
 - come back to what “multicore” actually means later on ...
- Connected together by a network



- Each laptop is called a *compute node*
 - each has its own operating system and network connection
- Suppose each node is a quadcore laptop
 - total system has 20 processor-cores

Typical HPC system layout

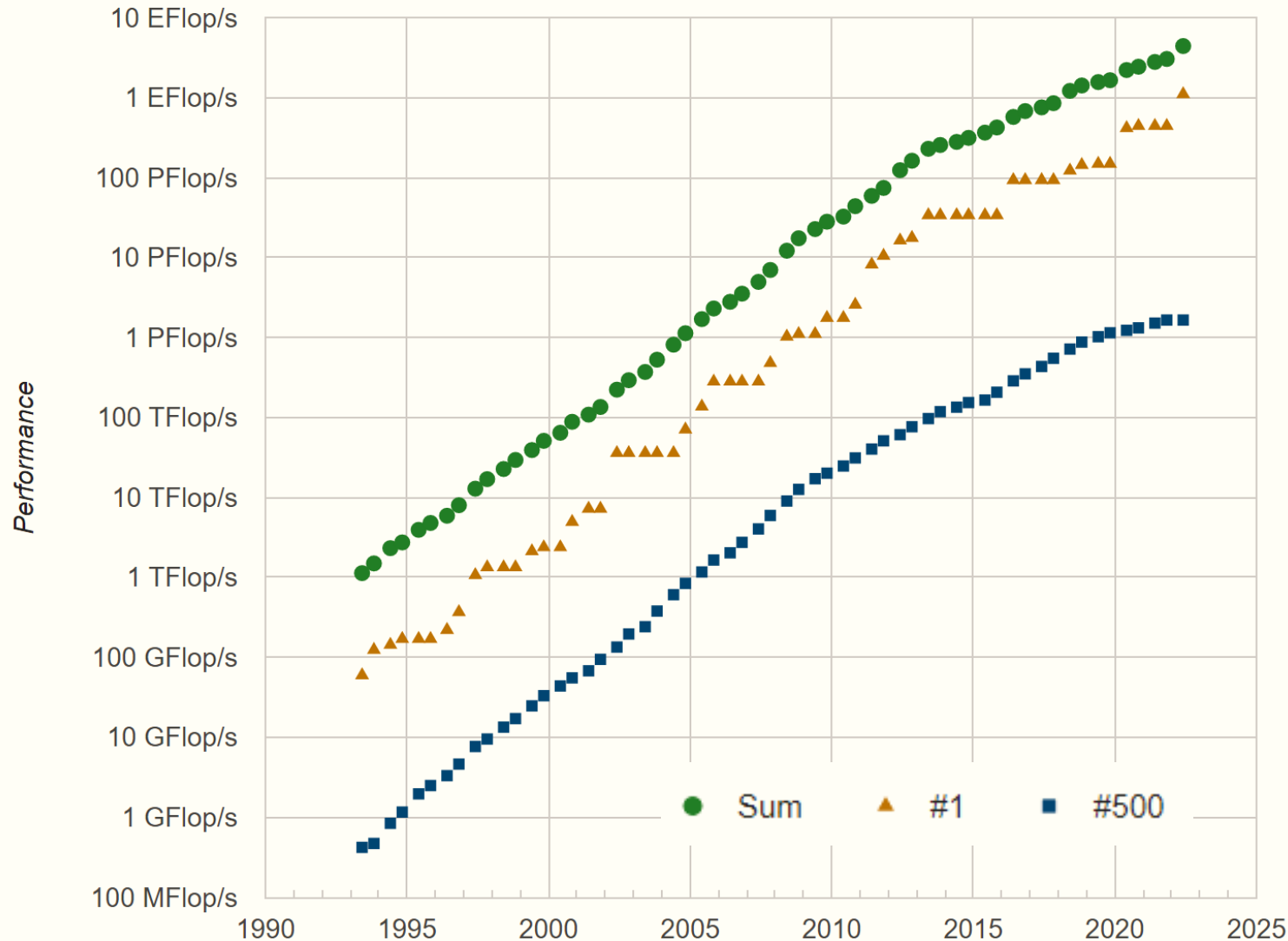


Top500 list: www.top500.org

- bi-annual competition to solve an enormous set of linear equations (Linpack benchmark)

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
6	HPC6 - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461

Performance Development



FLOPS

- Yotta: 10^{24}
- Zetta: 10^{21}
- Exa: 10^{18}
- Peta: 10^{15}
- Tera: 10^{12}
- Giga: 10^9
- Mega: 10^6
- Kilo: 10^3

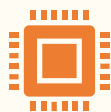
Source: Top500.org

El Capitan: #1 in the world (June 2025)



- HPE system owned by US Department of Energy
- 11,039,616 processor cores
 - mainly AMD Instinct MI300A GPUs
- 2.75 EFlops peak performance
 - Linpack runs at 1.74 Eflops
- 30MW power
 - 10x power of ARCHER2 but approaching 100x performance

But why?



Computational Simulation, mostly

Focus on science and the problem to be solved

- Simulation of scientific problem or environment
- Input of real data
- Output of simulated data
- Parameter space studies
- Wide range of approaches



Explore universe through simulation rather than experimentation

- Test theories
- Predict or validate experiments
- Simulate “untestable” science

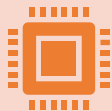


Reproduce “real world” in computers

- Generally simplified
- Dimensions and timescales restricted



HPC - Parallelism



Simulation
science drives
computing
power

Consistently need more
computation power than available
Runtime of months on a single
processor not uncommon
Parallel programs often start out as
serial programs



Why not just
make a faster
chip?

Worked in the past, but starting to
run into problems



Solution

Use many CPUs cooperatively on
the same problem
HPC computing synonymous with
parallelism

Quantifying Performance

Serial computing concerned with *complexity*

- how execution time varies with problem size N
- adding two arrays (or *vectors*) is $O(N)$
- matrix times vector is $O(N^2)$, matrix-matrix is $O(N^3)$

Look for clever algorithms

- naïve sort is $O(N^2)$
- divide-and-conquer approaches are $O(N \log(N))$

Parallel computing *also* concerned with scaling

- how time varies with number of processors P
- different algorithms can have different scaling behaviour
- but always remember that we are interested in minimum time!

Performance Measures

$T(N, P)$ is execution time for
size N on P processors

Speedup:

typically, $S(N, P) < P$

$$S(N, P) = \frac{T(N, 1)}{T(N, P)}$$

Parallel Efficiency:

typically, $E(N, P) < 1$

$$E(N, P) = \frac{S(N, P)}{P} = \frac{T(N, 1)}{PT(N, P)}$$

Serial Efficiency:

Typically, $E(N) \leq 1$

$$E(N) = \frac{T_{best}(N)}{T(N, 1)}$$

Parallel Scaling

Scaling describes how the runtime of a parallel application changes as the number of processors is increased

Can investigate two types of scaling:

Strong Scaling -
increasing P ,
constant N .



Problem size stays same as number of processors increase.

Decreasing the work per processor.

Weak Scaling -
increasing P ,
increasing N .



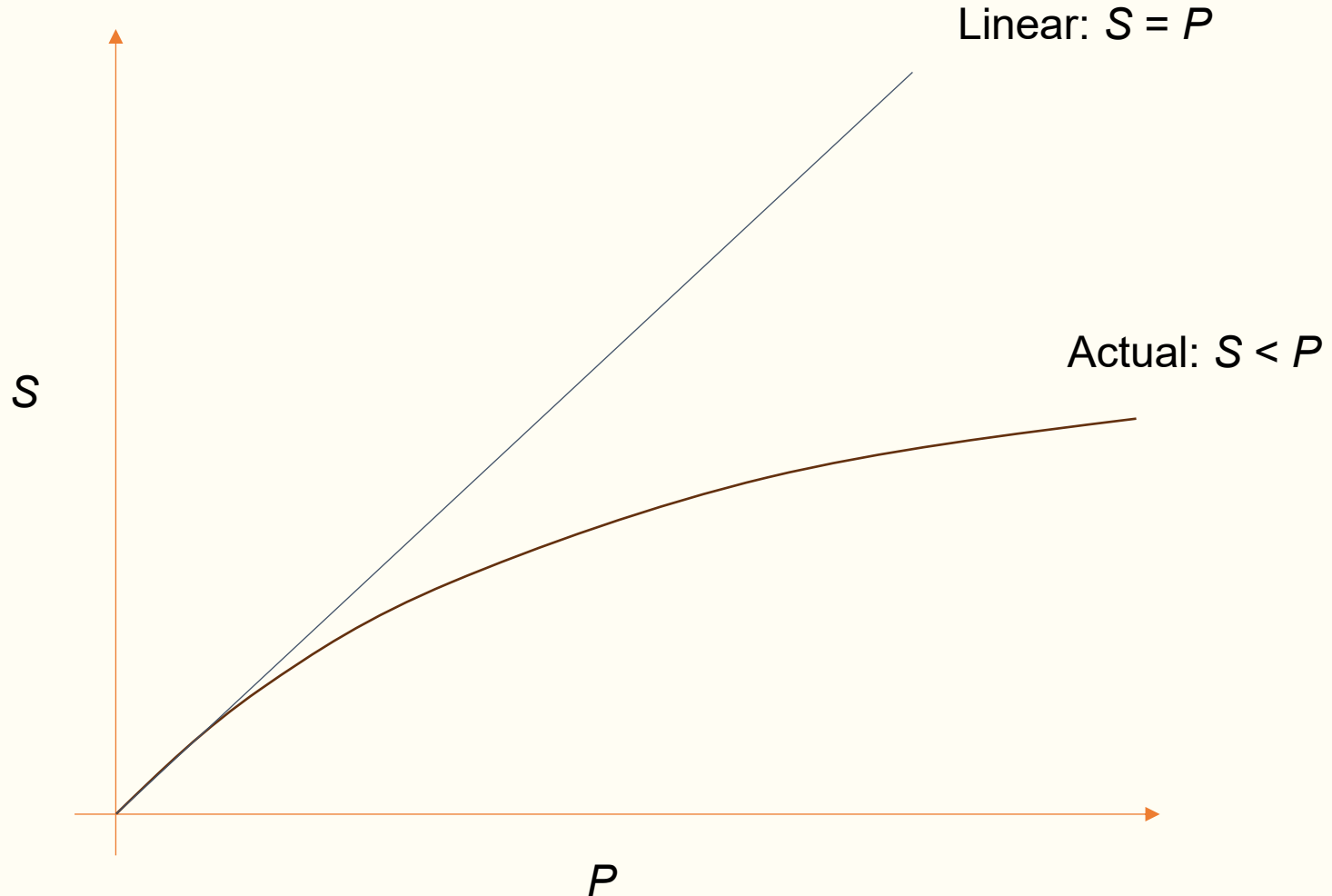
Problem size increases at same rate as the number of processors.

Work per processor stays the same.

Strong Scaling Example

- Have developed a parallel program
 - want to solve a given problem as fast as possible
- Calculation takes 12 hours on 1 processor
 - now run *the same problem* on 10 processors
 - would hope it takes 1.2 hours (speedup of 10)
- We measure runtime of 1.5 hours on 10 processors
 - actual speedup is $12/1.5 = 8$
 - parallelisation is not perfect
 - parallel efficiency = $8/10 = 0.8$ (80% of the best we could expect)
- But ... (and we normally ignore this part!)
 - original serial program took only 9 hours
 - serial efficiency is $9/12 = 0.75$ (serial code is 25% faster)

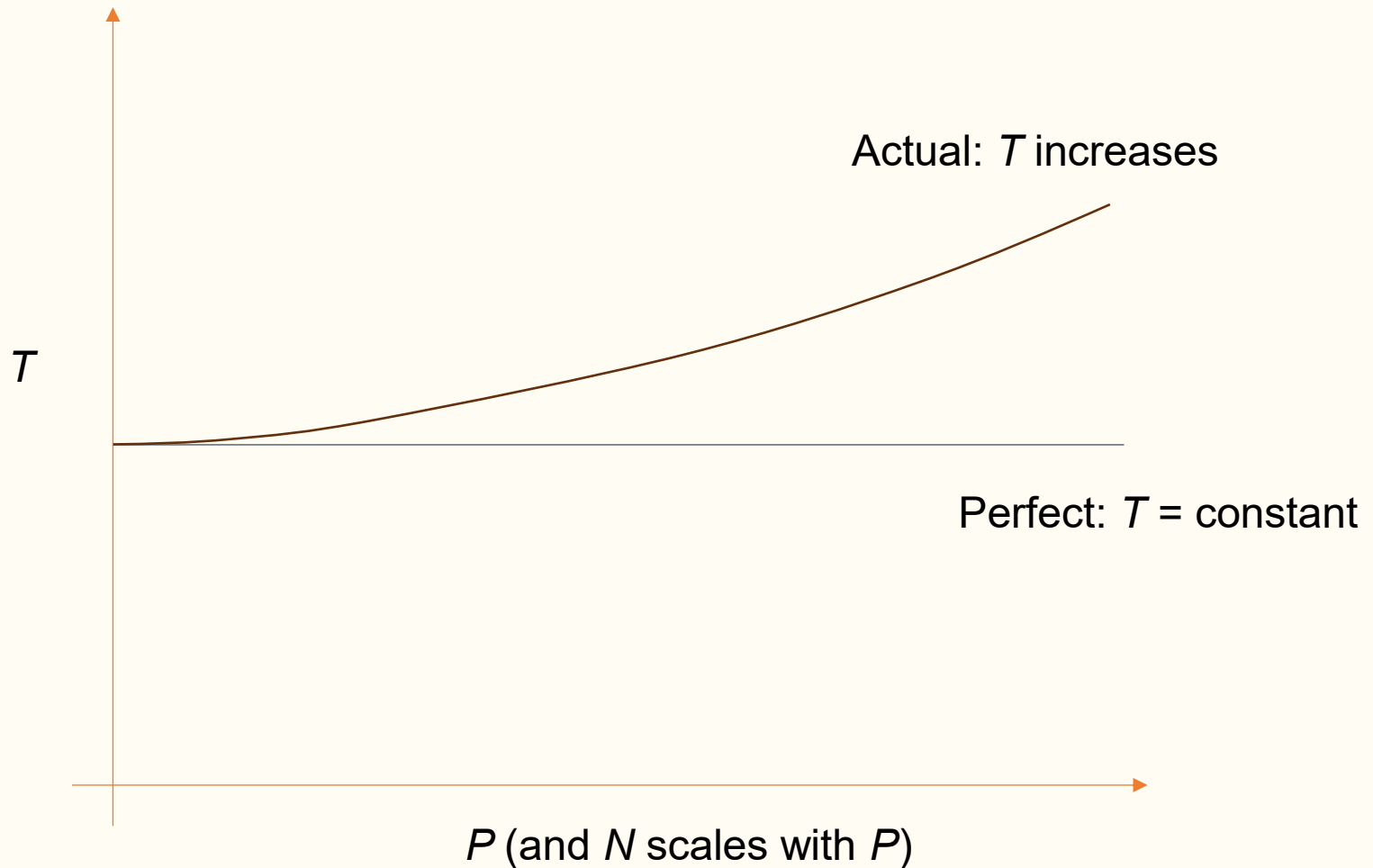
Typical strong scaling (“Amdahl’s law”)



Weak Scaling Example

- Have developed a parallel program
 - want to run bigger problems but in the same time
 - e.g. want to produce the best possible weather forecast for tomorrow
- Calculation takes 12 hours on 1 processor
 - Now multiple both P **and** N by 10
 - run a problem 10 times larger on 10 times as many processors
 - would hope it still takes 12 hours (constant time)
- We measure runtime of 15 hours on 10 processors
 - parallelisation is not perfect
 - overheads mean an increase in time

Typical weak scaling graph

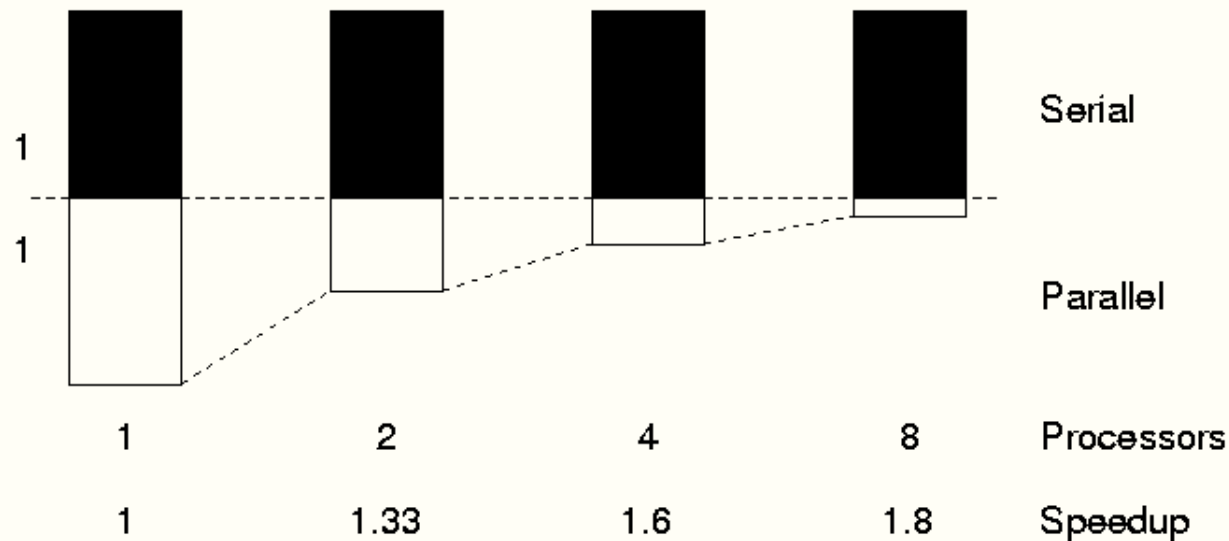


The Serial Component

Amdahl's law:

“the performance improvement to be gained by parallelisation is limited by the proportion of the code which is serial”

Gene Amdahl, 1967



Amdahl's law

Assume a fraction α is completely serial
- time is sum of serial and potentially parallel

Parallel time

$$T(N, P) = \alpha T(N, 1) + \frac{(1-\alpha)T(N, 1)}{P}$$

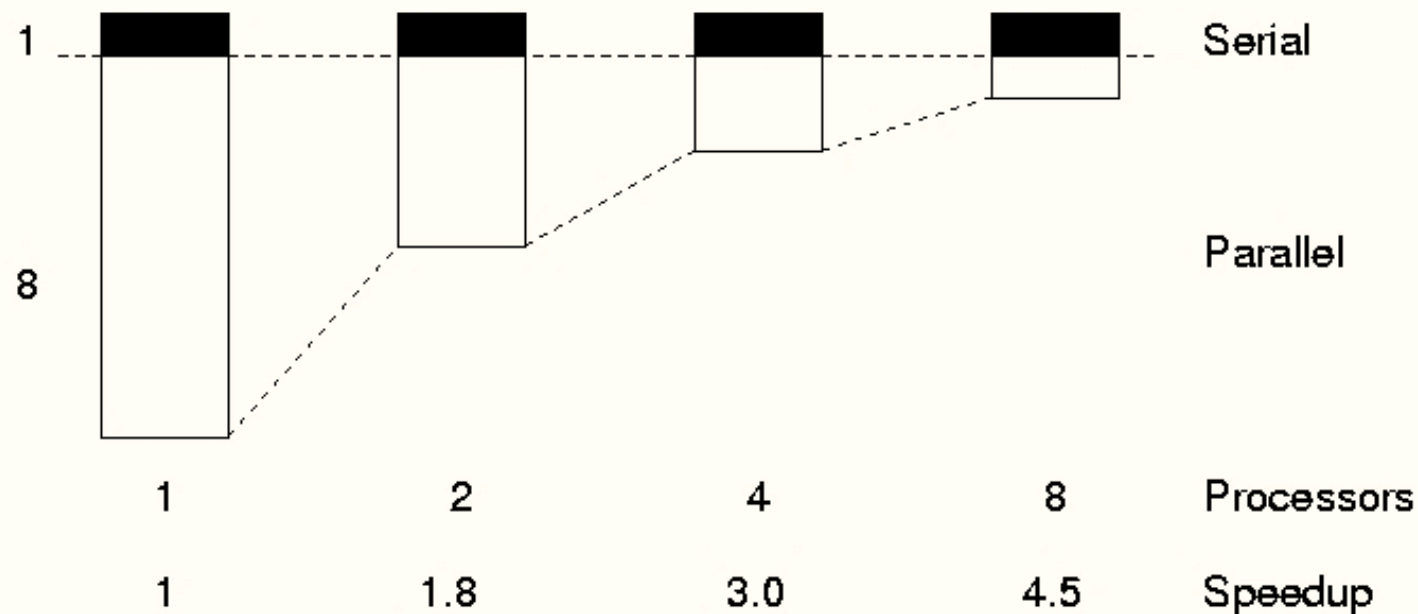
Parallel speedup

$$S(N, P) = \frac{T(N, 1)}{T(N, P)} = \frac{P}{\alpha P + (1 - \alpha)}$$

- for $\alpha = 0$, $S = P$ as expected (i.e. $E = 100\%$)
- otherwise, speedup limited by $1/\alpha$ for any P
- impossible to effectively utilise large parallel machines?

Gustafson's Law

Need larger problems for larger numbers of processors



Utilising Large Parallel Machines

Assume parallel part grows with N (i.e is $O(N)$); serial part constant with N (i.e. $O(1)$)

Time

$$\begin{aligned} T(N, P) &= T_{serial}(N, P) + T_{parallel}(N, P) \\ &= \alpha T(1, 1) + \frac{(1-\alpha)NT(1,1)}{P} \end{aligned}$$

Speedup

$$S(N, P) = \frac{T(N,1)}{T(N,P)} = \frac{\alpha + (1-\alpha)N}{\alpha + (1-\alpha)\frac{N}{P}}$$

Scale problem size with CPUs, ie set $N = P$ **Weak Scaling**

Speedup

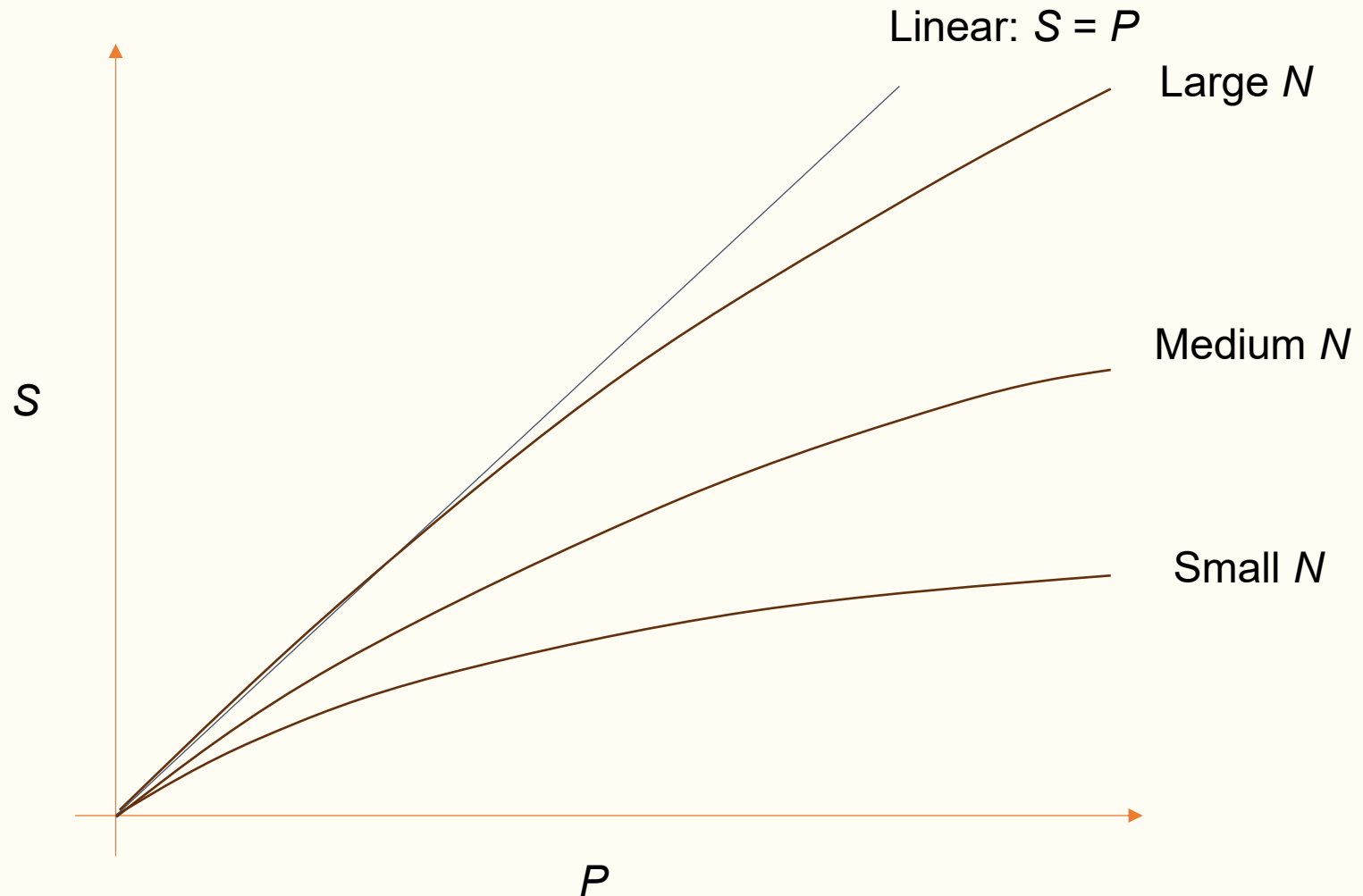
$$S(P, P) = \alpha + (1 - \alpha)P$$

Efficiency

$$E(P, P) = \frac{\alpha}{P} + (1 - \alpha)$$

Maintain constant efficiency $(1-\alpha)$ for large P

Strong scaling showing Gustafson's Law



Performance Summary

Definitions to Remember:

Speed-up equations

Efficiency equations

Amdahl's Law

Gustafson's Law

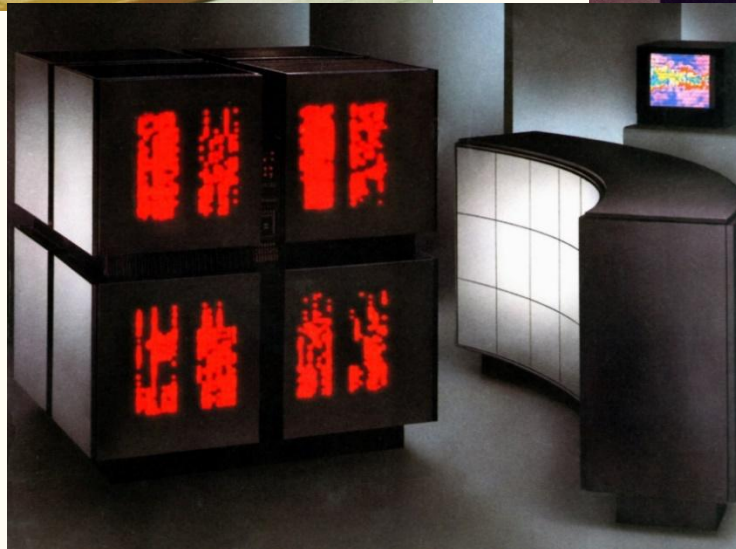
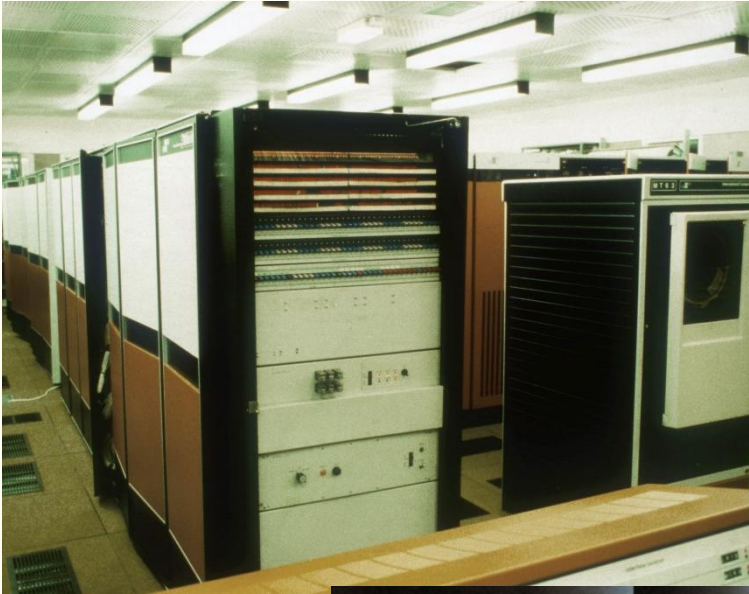
Parallel Computers at Edinburgh

- **Nice summary:**

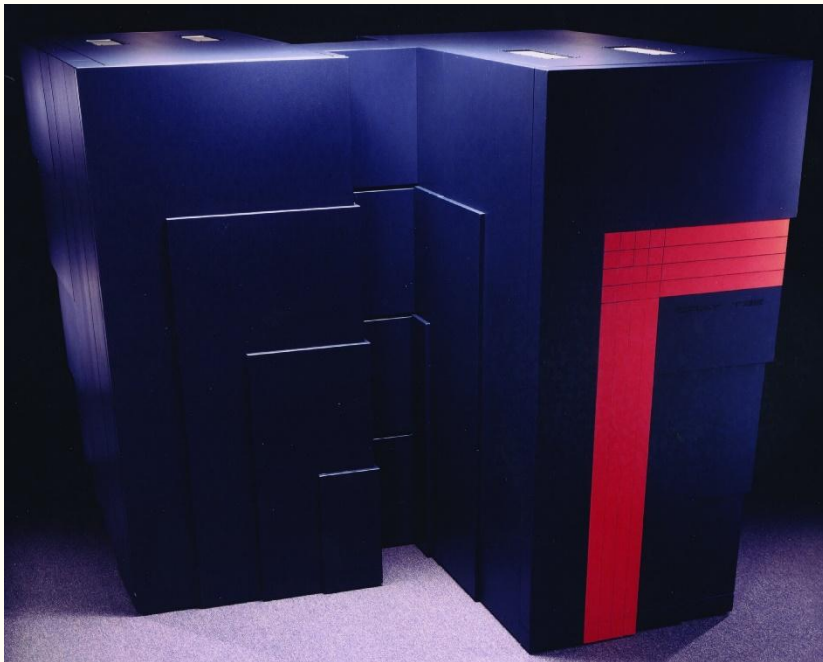
https://www.archer2.ac.uk/community/outreach/materials/history_of_machines

- 1981 ICL DAP (SIMD: 4,096 processors) - 0.03 GFlops
- 1991 TMC CM-200 (SIMD: 16,000 processors) 3.3 GFlops
- 1994 Cray T3D (MIMD-NUMA: 512 processors) + Cray Y-MP (Vector) - 51 GFlops
- 1997 Cray T3E (MIMD-NUMA: 344 processors) 118 GFlops
- 2005 IBM BlueGene/L (MIMD-DM: 2048 cores) - 5.6 TFlops
- 2007 Cray XE6 (MIMD-DM: 90,112 cores) - 55 TFlops
- 2013 Cray XC30 (MIMD-DM: 118,080 cores) – ARCHER 2.5 PFlops
- 2016 HPE/SGI 8600 (MIMD-DM: 10,080 cores + GPU) – Cirrus
- 2021 HPE Cray Ex/Shasta (MIMD-DM: 748,544 cores) – ARCHER2 20 PFlops
- 2027 (?) Next UK national supercomputer – probably GPUs, perhaps 1 EFlops?

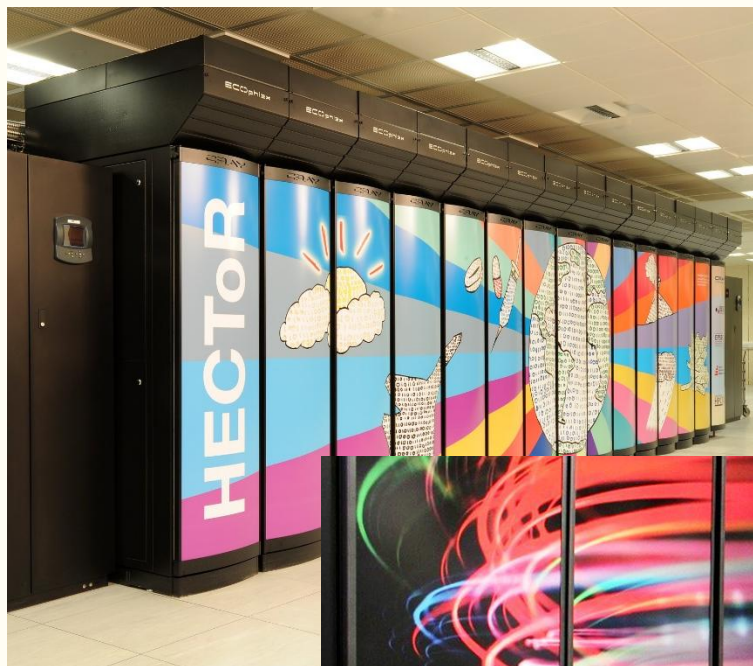
Early Parallel Computers at Edinburgh



More Parallel Computers at Edinburgh



Recent Computers at Edinburgh



HPE Cray EX (Cray Shasta)



Facts and Figures

Lifetime: 2020 - ?

5,848 nodes, each with $2 \times$ AMD Zen2 7742, 2.25 GHz, 64-core CPU

Total 748,544 cores in 23 cabinets

Peak Performance: 28 PFLOP/s (estimated)

Architecture: MIMD-DM

Memory: 1.5 PetaBytes

Programming: MPI

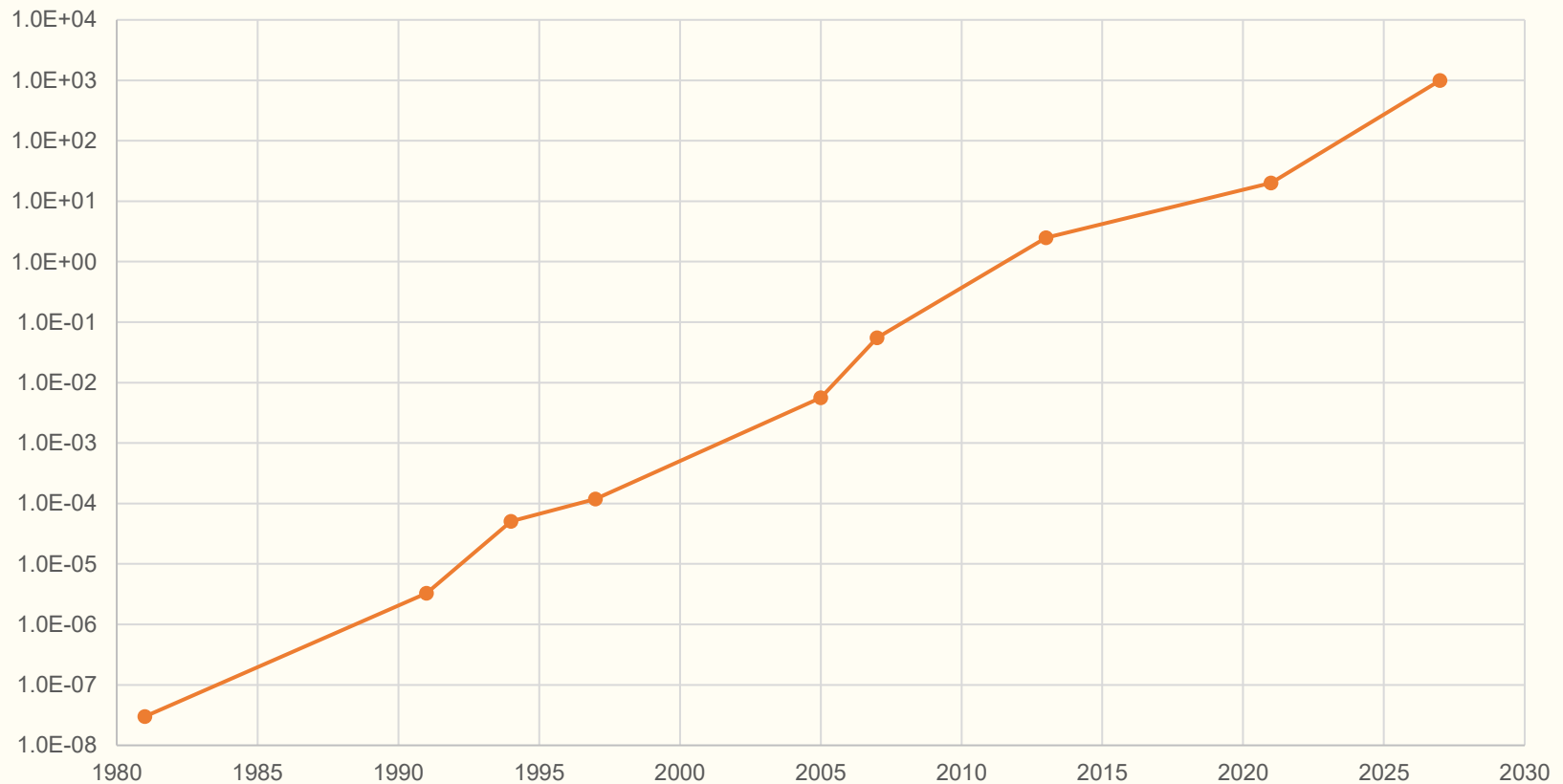
Notes

“Slingshot” interconnect topology

Same base design as El Capitan, but no GPUs!

Performance over time

EPCC Linpack PFlops



Summary



Parallel computing started as a range of weird and wacky architectures

Each with their own unique languages, OS, tools, etc. each required substantial investment of time to port to



Standardisation of programming paradigms was vital for parallel computing to mature

MPI, OpenMP,...



Edinburgh/EPCC has been at the forefront of parallel computing and HPC for 30 years



The future appears extremely exciting for HPC, computational science, data science, AI and EPCC